



Handling Catastrophic Forgetting: Online Continual Learning for Next Activity Prediction

Tamara Verbeek and Marwan Hassani^(✉) 

Eindhoven University of Technology, Eindhoven, The Netherlands
t.a.m.verbeek@student.tue.nl, m.hassani@tue.nl

Abstract. Predictive business process monitoring focuses on predicting future process trajectories, including next-activity predictions. This is crucial in dynamic environments where processes change or face uncertainty. However, current frameworks often assume a static environment, overlooking dynamic characteristics and concept drifts. This results in *catastrophic forgetting*, where training while focusing merely on new data distribution negatively impacts the performance on previously learned data distributions. Continual learning addresses, among others, the challenges related to mitigating catastrophic forgetting. This paper proposes a novel approach called Continual Next Activity Prediction with Prompts (CNAPwP) which adapts the DualPrompt algorithm for next-activity prediction to improve accuracy and adaptability while mitigating catastrophic forgetting. New datasets with recurring concept drifts are introduced, alongside a task-specific forgetting metric that measures the prediction accuracy gap between initial and subsequent task encounters. Extensive testing on both synthetic and real-world datasets shows that this approach outperforms five competing methods, demonstrating its potential applicability in real-world scenarios. An open source implementation of our method together with datasets and results are available under: <https://github.com/TamaraVerbeek/CNAPwP>.

Keywords: Process mining · Next-activity prediction · Concept drift · Catastrophic forgetting · Continual learning

1 Introduction

Recent emphasis on predictive process monitoring highlights its critical importance by enabling organizations to predict the future trajectories of individual process instances dynamically [2, 3]. Predictive business process monitoring offers decision support that empowers organizations to make adjustments to their processes. In predictive process monitoring, next activity prediction holds significant relevance due to its ability to anticipate and plan for future actions within business processes. This predictive capability enables proactive management of resources, ensuring they are deployed effectively to meet upcoming demands. For example, accurate next activity prediction in customer service enables organizations to plan ahead, optimize resources, and reduce response times, leading to more efficient operations and improved customer satisfaction [23].

Offline predictive process monitoring involves two main phases: training and testing. Performing next activity prediction in an online setting offers distinct advantages over

offline methods. In an online environment, predictive models continuously learn and adapt in real-time as new data streams in, more accurately reflecting the current state of operations. This dynamic approach allows organizations to respond swiftly to changing conditions and optimize processes continuously, whereas offline methods typically rely on static datasets that may not capture evolving patterns and trends effectively [6,9,23].

A significant challenge in online next activity prediction arises from adapting to new concepts following shifts in data distribution. While real-time model updates enable responsiveness to evolving trends and operational changes, they also introduce the risk of *catastrophic forgetting* [8]. This phenomenon occurs when the prediction model overwrites previously learned information with a new data distribution, potentially leading to a loss of accuracy or reliability in predicting next activities when refacing previously learned data distributions. Balancing dynamic model updates with the retention of critical historical knowledge is crucial, especially given constraints such as storage limitations and privacy concerns that prevent organizations from retaining all historical data indefinitely. Developing robust online prediction models requires efficient use of historical data while integrating new information. In process mining, recurring drifts from factors such as seasonal changes or shifts in customer behavior make it essential to prevent catastrophic forgetting. Neglecting recurring tasks would compromise the model’s long-term performance.

Only recently algorithms have been developed for online predictive process monitoring [7] and specifically for online next activity prediction [9,23]. In contrast, numerous algorithms already exist for online image classification that avoid catastrophic forgetting, including [8,17,21,22]. By leveraging the techniques utilized in such algorithms, there is potential to create high-performance online next activity prediction models for this purpose. Unlike independent data points in classification, next activity prediction requires capturing temporal dependencies, which presents unique challenges.

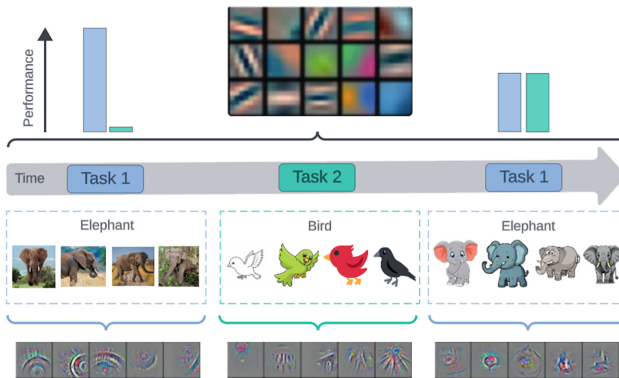


Fig. 1. A continual learning setup where two tasks are sequentially learned. The second appearance of Task 1 implies a domain change from photographs to drawings.

Continual learning, also known as lifelong learning, plays a crucial role in enhancing the effectiveness of online prediction models. This concept facilitates the retention

of previously learned information while integrating new insights, thereby maintaining a comprehensive understanding of the data distribution landscape. By leveraging task-specific and task-invariant features, continual learning helps to mitigate catastrophic forgetting. Figure 1 illustrates a stream of animal image classification tasks. Initially, the model classifies elephant photographs, followed by a task classifying bird drawings. Typically, an online model forgets how to classify elephants when switching to birds. In the performance depicted for the occurrences of *Task 1*, we observe a decline after learning *Task 2* when the model is compelled to deal again with the first task. However, by identifying task-invariant features, such as distinguishing between photographs and drawings, we have the potential to preserve valuable information for the subsequent reappearance of the initial task, for instance when involving elephants depicted in drawings. By understanding the distinctions between photographs and drawings and incorporating task-specific insights about elephants, we improve the model’s ability to classify representations of elephants in drawings accurately. These features can be incorporated as prompts. A prompt is an additional piece of information provided to the model to guide its learning or inference process. By using prompts that highlight task-specific and task-invariant features, we can maintain the contextual knowledge necessary for effectively addressing evolving tasks, ensuring continued success in classification, particularly when faced with domain shifts in a task.

In this paper, we propose a framework called Continual Next-Activity Prediction with Prompts (CNAPwP) with components for handling temporal dependencies, capturing long-term patterns, and incorporating memory and context into the learning process. The key contributions of this work include: (i) introducing an innovative framework that combines general and expert prompts for dynamic adaptation and learning from streaming data, (ii) developing the “task-specific forgetting” evaluation metric to assess the retention and the degradation of task-specific knowledge, (iii) creating new datasets that are specifically tailored for evaluating online predictive process monitoring, and, (iv) conducting a comprehensive experimental evaluation which demonstrates that CNAPwP significantly outperforms existing methods in dynamic environments.

The remainder of this paper is structured as follows: in Sect. 2, we review related literature; in Sect. 3, we explain foundational concepts and define the problem; in Sect. 4, we present our framework; in Sect. 5, we discuss evaluation criteria and analyze results; and finally, we conclude with remarks in Sect. 6.

2 Related Work

At its core, continual learning seeks to equip machine learning models with the ability to continuously learn and adapt over time, similar to how humans acquire and refine knowledge throughout their lives. Within this framework, researchers have explored various methodologies, each offering unique perspectives and strategies. These approaches include memory-based, architecture-based, regularization-based, and prompt-based methods.

Memory-based approaches, inspired by human cognition and memory systems, focus on storing and retrieving past experiences or knowledge. These methods can be broadly classified into two categories. The first involves retaining actual past experiences, as demonstrated by techniques such as Experience Replay [17], iCaRL [16],

DynaTrainCDD [9], Maximally Interfered Retrieval [15], and Gradient Episodic Memory [12]. The second category, on the other hand, generates past experiences during training, with Generative Replay [10] serving as a key example.

Regularization-based methods aim to prevent catastrophic forgetting by constraining weight updates during training. This constraint is typically achieved by evaluating the importance of each parameter for previously learned tasks. For example, Elastic Weight Consolidation [8] determines parameter significance, while Synaptic Intelligence [24] imposes penalties during the training of new tasks to preserve essential knowledge. Alternatively, the importance of parameters can be assessed based on their impact on output sensitivity, with selective penalties applied to critical parameters to mitigate forgetting, as seen in Learning without Forgetting [11].

Architecture-based approaches prioritize adjusting the neural network’s structure to integrate new data while preserving existing knowledge. One approach involves dynamic architectures, which expand the network by adding more neurons or layers for each task. This allows the model to continuously grow and adapt without forgetting previous knowledge, as exemplified by methods such as Progressive Neural Networks [18] and Dynamic Neural Networks [5].

Prompt-based approaches, a more recent addition to the continuum, introduce a novel perspective on continual learning challenges. These methods entail attaching static or adaptable “instructions”, also referred to as prompts, to direct the model’s behavior. These prompts can take various forms, such as specific input patterns, embeddings, or task-specific tokens, and they help the model recall and apply knowledge from earlier tasks. Examples of prompt-based approaches include Learning to Prompt [22] and DualPrompt [21].

We will delve deeper into DualPrompt [21], since our approach adopts a modified version of it. DualPrompt is a continual image classification method, inspired by prompting techniques commonly used in natural language processing to guide models toward desired results [21]. This approach is particularly useful for sequence-like data, as it helps the model understand the underlying structure and the relationships within the sequences. It comprises two key components: G-Prompt and E-Prompt, which learn task-invariant and task-specific knowledge, respectively. The architecture of DualPrompt involves multiple Multi-Head Self-Attention (MSA) layers, with G-Prompt and E-Prompt integrated into these layers. The **G-Prompt** is a shared parameter for all tasks, applied during both training and testing to capture general knowledge. The **E-Prompt** on the other hand contains a set of task-specific parameters, each linked to a learnable key representing the task’s unique features. These prompts are strategically attached to different layers of the model, with G-Prompt typically at shallower layers and E-Prompt at deeper layers. This setup leverages the varying levels of feature abstraction in the layers. Two prompting functions can be used to combine the prompts with embedding features. In **Prompt Tuning**, prompts are attached to the input tokens of each layer, increasing the output length. In **Prefix Tuning**, the prompt is split and attached to specific tokens, maintaining the same output length as the input. During inference, an input is transformed to identify the closest task key and the associated E-Prompt using cosine similarity. The shared G-Prompt and matched E-Prompt are then

attached to multiple MSA layers to produce the model's output. During training, the E-Prompt and G-Prompt are jointly trained with the classifier based on the task identity.

TFCLPM [20] is the first continual learning model specifically designed for predictive process mining. It utilizes a memory-based approach to store hard samples and a dynamic loss to predict next activities while mitigating catastrophic forgetting.

Outside continual learning, innovative methodologies have emerged in offline next-event prediction. The state of the art in offline next-activity prediction is evolving through advancements in deep learning architectures, attention mechanisms, hybrid modeling approaches, and the integration of Generative Adversarial Networks (GANs), enhancing sequence generation and capturing complex temporal dependencies [19]. Studies explore incremental techniques to update predictive models with new process execution data. Pauwels et al. [14] compare various update strategies, including re-training with and without hyperoptimization and incremental updates, demonstrating the effectiveness of incremental updates in maintaining model quality while offering real-time adaptability.

To efficiently store traces, prefix trees [6] offer a robust solution. These trees, represented as simple graphs, organize event sequences efficiently, with each node representing an event and edges connecting sequential events. As events occur, nodes are either created or their frequencies updated, capturing unique event sequences for individual cases.

3 Preliminaries and Problem Formulation

In this section we delve into formulating the problem of utilizing continual learning in process prediction. We aim to develop an algorithm that continuously processes an event stream $S = \{e_1, e_2, \dots\}$ as events are generated, where an event $e = (c, a, t, v_1, \dots, v_A)$ is a tuple of case identifier c , activity label a , timestamp t , and the values of other event attributes $v_1, \dots, v_A$. A case refers to a single instance of the process being analyzed or executed, encompassing all events, their attributes, and contextual information associated with that instance. A trace $\sigma^{(i)} = \langle e_{1i}, \dots, e_{ni} \rangle$ denotes any finite sequence over the set of all events. Without loss of generality and since we are dealing merely with event logs, the terms trace and case are used interchangeably in the remainder of the paper and refer to the sequence of activities within the instance. A stream S contains multiple traces.

Given $\sigma^{(i)}$, the prefix represents the sequence of activities executed up to a certain point in a trace's lifecycle. The prefix of length k is defined by $\sigma_{\leq k}^{(i)} = \langle e_{1i}, \dots, e_{ki} \rangle$. On the other hand, a suffix refers to the part of a trace that occurs after a particular event in that trace. Given $\sigma^{(i)}$, the suffix of events of length k is defined by $\sigma_{> n-k}^{(i)} = \langle e_{(n-k)i}, \dots, e_{ni} \rangle$. For each event e that occurs in stream S , we want to predict the next activity name of event $e_{(k+1)i}$ happening at the beginning of suffix $\sigma_{> n-k}^{(i)}$ based on the prefix $\sigma_{\leq k}^{(i)}$.

Definition 1 (Next activity prediction). *Let there be a sample of prefixes of sequences $\mathcal{P} = \{\sigma_{\leq k}^{(i)}\}_{i=1}^{i=m}$ where $2 \leq k < |\sigma^{(i)}|$ is the prefix length and m is the sample size.*

Given a prefix of an events sequence $\sigma_{\leq k}^{(i)}$, the next activity prediction is $\hat{a}_{(k+1)i}$ of activity $a_{(k+1)i}$ happening at the beginning of suffix $\sigma_{>n-k}^{(i)}$.

To continuously predict the next activity in a stream, we must engage in online next activity prediction.

Definition 2 (Online Next Activity Prediction). Assume we aim to perform ongoing predictions of the next activity on a stream of events. Upon the arrival of each new event e_{ki} , we continuously utilize the prefix $\sigma_{\leq k}^{(i)}$ to predict the subsequent activity $a_{(k+1)i}$.

A stream S encompasses multiple learning tasks, each having several traces.

Definition 3 (Learning Task). A model will learn a learning task $\mathcal{T}_n$ with a corresponding dataset $\mathcal{D}_n$ for $n \in [1, \dots, \mathcal{N}]$ where $\mathcal{N}$ is the number of learning tasks. Let $\mathcal{T}_p$ and $\mathcal{T}_r$ represent two distinct learning tasks, where $p \neq r$. This indicates that $\mathcal{T}_p$ and $\mathcal{T}_r$ belong to different processes or data, implying there is no relationship between these learning tasks.

In the context of a stream of events involving multiple learning tasks, the distribution evolves over time, necessitating continuous updates to the prediction model. Within a learning task, the domain may change. In such cases, we seek a prediction model that can adapt to this shift, enabling domain incremental learning.

Definition 4 (Domain Incremental Learning). In domain incremental learning, all tasks share the same set of outputs $\mathcal{Y}$, i.e., $\mathcal{Y}_n = \mathcal{Y}_1 \forall n \in \mathcal{N}$. The objective is to learn all tasks while the joint probability between input and output changes.

Domain incremental learning focuses on adapting the prediction model to variations within the same task. However, in many scenarios, the model must also handle entirely new tasks that are distinct from previously encountered ones. This necessitates a special form of incremental learning, known as *task incremental learning*.

Definition 5 (Task Incremental Learning). In task incremental learning, two tasks $\mathcal{T}_n, \mathcal{T}_m$ where $n, m \in [1, \dots, \mathcal{N}]$ have no correspondence to each other if $n \neq m$. Each task has unique objectives and is associated with a separate process, necessitating the model to acquire new patterns, features, or behaviors.

Changes in the domain and the emergence of new tasks both result from concept drifts.

Definition 6 (Concept Drift [4]). A concept drift between timestamps t_0 and t_1 is defined as $\exists X : p_{t_0}(X, y) \neq p_{t_1}(X, y)$ where p_{t_0} denotes the joint distribution at time t_0 between the set of input variables X and the target variable y .

A task repeating multiple times is called a recurrent concept drift.

Definition 7 (Recurrent Concept Drifts). A recurrent concept drift implies repeated changes in tasks, with the possibility of cycles that bring the system back to the original task.

When updating the model after a concept drift, the goal is to utilize as much data as possible. If a task reappears, it is advantageous to have stored data about this task, ensuring that it is not forgotten. However, this poses challenges such as storage limitations or privacy concerns. This often prevents keeping all past data, but relying only on recent data introduces bias and causes catastrophic forgetting [1].

Definition 8 (Catastrophic Forgetting [8]). *Let there be a model at any point in time that has learned a sequence of $\mathcal{T}$ learning tasks. When faced with the $(\mathcal{T} + 1)$ th task, the model tends to forget how to predict the next activities of the $\mathcal{T}$ previously learned tasks.*

This phenomenon poses a significant challenge in dynamic environments. To address the issue of catastrophic forgetting, continual learning enables models to retain and incorporate knowledge from previous tasks while learning new ones.

Definition 9 (Continual Learning [9]). *Let there be a model at any point in time that has learned a sequence of $\mathcal{T}$ learning tasks. When faced with the $(\mathcal{T} + 1)$ th task, the model can leverage the past knowledge in the knowledge base to help learn this task. The objective is to optimize the performance of the new task while minimizing the decrease in performance on the previous tasks.*

By compiling the definitions, we arrive at the following definition for a continual learning algorithm used to predict the next activity.

Definition 10 (Continual Learning for Next Activity Prediction). *Consider a model that has learned to predict the next activities for an event stream S . Throughout S , concept drifts may occur which refer to the alteration in the learning task. The event stream contains a sequence of $\mathcal{T}$ learning tasks. A learning task for next activity prediction is defined by a sample of prefixes of sequences $\mathcal{P}$. Given a prefix of an events sequence $\sigma_{\leq k}^{(i)}$, the model has to predict the activity $\hat{a}_{(k+1)i}$ occurring at the beginning of the suffix $\sigma_{>n-k}^{(i)}$.*

When presented with the $(\mathcal{T} + 1)$ th task, the model must accurately predict the activities for this new task without suffering from catastrophic forgetting.

The solution to the challenges of continual learning in process prediction models must meet several key requirements. Firstly, it should efficiently utilize an input event stream S consisting of multiple learning tasks $\mathcal{T}$, as defined in Definition 3, and predict next activities accurately in near-real-time, as described in Definition 2. Secondly, the solution should accommodate both domain and task incremental learning, mentioned in Definitions 4 and 5, respectively, allowing for seamless integration of shifts in the domain and the emergence of new tasks. Additionally, the solution must exhibit the capability to perform next activity predictions while remaining robust to concept drifts, mentioned in Definition 6. Moreover, the solution should aim to reduce catastrophic forgetting, as defined in Definition 8, particularly when tasks recur in the stream, as defined in Definition 7. In essence, the algorithm should incorporate continual learning for predicting next activities, as defined in Definition 10.

4 Continual Next-Activity Prediction with Prompts (CNAPwP)

In the section, we provide an overview of the CNAPwP architecture by detailing its components. The implementation is available on [github](https://github.com/TamaraVerbeek/CNAPwP)¹. Figure 2 shows the framework architecture for training and testing the prediction model.

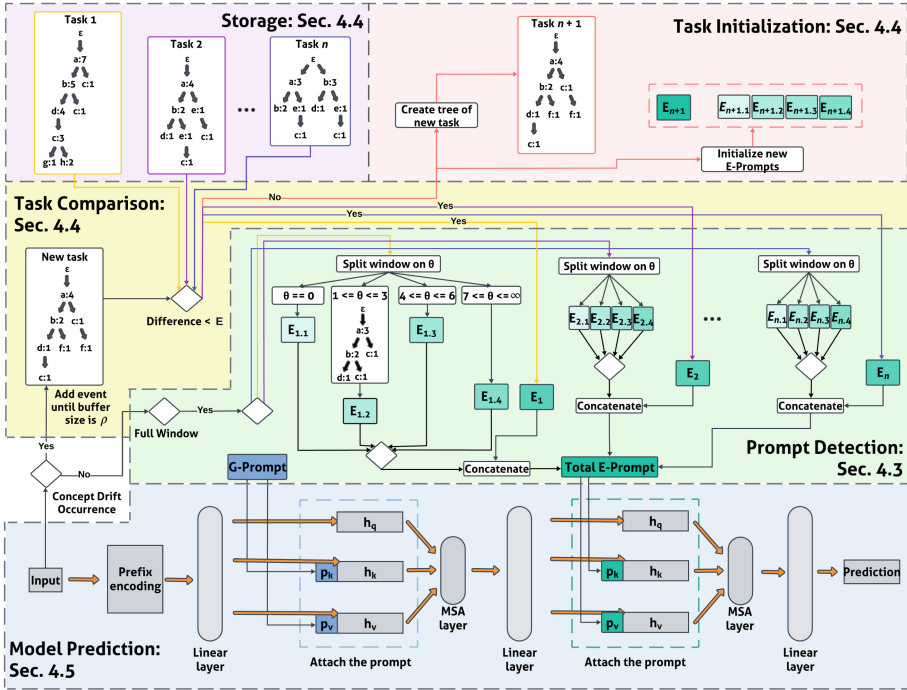


Fig. 2. Framework for Continual Next-Activity Prediction with Prompts (CNAPwP).

4.1 Preprocessing

Preprocessing involves **initial data reduction**, where unnecessary columns are removed from the event log to reduce noise, focusing on essential attributes such as case, event, resource, and completion time. If events are missing key attributes, the corresponding cases are excluded from the dataset. Attributes representing the resource should be retained if present. Next, **data is ordered chronologically** to represent a real-time data stream, and afterward, the completion time attribute is deleted. Next, for each new event, **a prefix is generated**. The approach checks whether the case has appeared within the window of events. If the case is found and there are sufficient events to match the prefix size, the prefix is formed. If there are insufficient events, the prefix is padded with “None” values. If the case is new and not previously encountered, a prefix composed entirely of “None” values is created. Once the prefix is defined, the next activity

¹ <https://github.com/TamaraVerbeek/CNAPwP>.

is assigned as the new event. Events outside the defined window are disregarded in this process. After generating the prefixes, the next step involves **encoding** both the prefix and the next activity. The prefix is one-hot encoded, generating sparse input vectors that are effective for handling high-dimensional categorical data, whereas the next activity is ordinally encoded, offering a compact and efficient representation.

4.2 Window Handling

The process starts by importing an event log and extracting key attributes necessary for predicting the next activity. These attributes are converted into a continuous event flow, transmitting events until the entire log is analyzed. Events are sequenced by activity completion order, allowing new cases to begin while others are still ongoing. As each event arrives, it is added to a dynamic window that accumulates events until it reaches its capacity, defined by γ . The optimal value of this parameter is established based on the experiment outlined in Sect. 5.4. Once full, the window operates on a ‘first-in, first-out’ basis, maintaining a consistent size to manage the sequence of events effectively.

4.3 Window Updating and Prompt Detection

When γ events accumulate, an update is initiated, as depicted in the green segment of Fig. 2. This ensures the model remains current with the evolving event stream, maintaining a dynamic framework ready to process new events seamlessly. After accumulating γ events, the data is partitioned into buckets based on prefix lengths θ . Each bucket represents a specific range of prefix lengths, ensuring a uniform distribution. Figure 2 illustrates that there are four buckets, with the first containing solely empty prefixes and the second containing prefixes ranging in length from 1 to 3, for example. During initialization, bucket ranges are selected based on optimal values. Data of a specific prefix length is added to buckets until a significant discrepancy occurs, ensuring each bucket has a comparable number of values. With the window data segmented into buckets, the next phase updates all method parameters, including the G-Prompt, the bucket-specific E-Prompts, the task-specific E-Prompt, and the model weights. Data is batched by bucket, with each batch containing data from a single bucket. Data from each bucket is used solely for updating the parameters within its corresponding bucket-specific E-Prompt. The task-specific E-Prompt, G-Prompt, and weights receive updates across all batches. The cross-entropy loss is used for updating the weights and parameters of the G-Prompt and E-Prompt:

$$\text{CrossEntropyLoss} = - \sum_{i=1}^N y_i \log(\hat{y}_i) \quad (1)$$

where N is the number of classes, y_i and $\hat{y}_i$ are the true and predicted probabilities for class i , respectively. Both the G-Prompt and E-Prompt are concatenated with the model to facilitate these targeted updates. An ablation study experiment, detailed in Sect. 5.5, is conducted to validate the use of both the G-Prompt and E-Prompt.

4.4 Concept Drift Detection

Routine weight and parameter updates continue until a concept drift is detected. CNAPwP assumes these shifts are anticipated, triggering a task recognition mechanism upon encountering a drift index. After a concept drift, a specified number of events ρ is stored in a prefix tree, as depicted in the yellow segment of Fig. 2. The optimal value of ρ is established through an experiment outlined in Sect. 5.4. The prefix tree of the new task is compared with the prefix trees of stored tasks, as shown in the purple section of Fig. 2. The comparison focuses on identifying overlaps and unique sequences between them. A high degree of overlap and fewer unique sequences in the new task's prefix tree indicate greater similarity between the trees. A dissimilarity ratio is then calculated to measure how distinct the tasks are, with the lowest ratio indicating the stored task most similar to the new task. For each dataset, an $\mathcal{E}$ value is established according to the dissimilarity between its constituent tasks. If the similarity between the new task and any of the stored tasks is less than $\mathcal{E}$, the model will utilize the E-Prompts associated with that task. The optimal value for $\mathcal{E}$ for each dataset is determined in an experiment outlined in Sect. 5.4.

If none of the stored tasks are similar to the new task, our model does not overfit to only previously seen tasks and, as such, initializes parameters for the new task, as depicted in the pink segment of Fig. 2. This involves setting up the E-Prompt parameters for the new task and each bucket, which are initialized randomly. A new prefix tree is also created to characterize the task and is stored together with the tasks already in storage. As events stream in, they are added to the prefix tree until it reaches 500 events, enough to capture the task's essence for future comparisons.

4.5 Prediction Model

A Multi-Head Self-Attention mechanism is utilized for next activity predictions. The Multi-Head Self-Attention mechanism enhances next activity prediction by capturing complex dependencies and relationships between events, allowing the model to weigh the significance of past activities effectively. This approach enables the model to handle variable-length sequences efficiently while extracting diverse features, leading to more accurate predictions based on historical patterns. The prediction model of the approach is depicted in the blue segment of Fig. 2.

The prefix of each input event from the event stream is used as the model input. During preprocessing, each event's prefix is converted into a one-hot encoded array with dimensions $[k, num_events]$, where k is the prefix length and num_events is the number of distinct events. Therefore, the encoded prefix of the event is initially determined. Subsequently, this encoded prefix undergoes a series of layers, ultimately resulting in a prediction of the next activity. The model consists of duplicated linear layers and Multi-Head Self-Attention layers. Linear layers take inputs of size $input_size$ and produce outputs of $3 \times input_size$, allowing prompt attachment via Prefix Tuning. For further details on Prefix Tuning, please refer to [21]. These outputs are then processed by a Multi-Head Self-Attention layer with softmax activation and dropout regularization. Outputs from the Multi-Head Self-Attention layers feed into a dense layer with matching input and output sizes, equivalent to the number of events times the prefix length.

Finally, the output passes through a softmax layer, converting it into a probability distribution over possible activities. The activity with the highest probability is predicted as the next activity.

During inference, the G-Prompt and E-Prompt are appended to the input of the Multi-Head Self-Attention layers. The G-Prompt remains consistent across all inputs, while the E-Prompt is determined by the task recognition mechanism. The task linked with the input event is identifiable due to the mechanism activated after a concept drift, as explained in Sect. 4.4. The bucket-specific E-Prompt is chosen based on the prefix size σ of the input event. This bucket-specific E-Prompt is combined with the task-specific E-Prompt and applied to the input using a designated prompting function, such as Prefix Tuning. The justification of selecting the prompting function is based on the related experiment described in Sect. 5.4.

5 Experimental Evaluation

5.1 Evaluation Metrics

Typically, studies rely on metrics such as accuracy and running time to assess performance. However, these metrics often fall short of capturing the longitudinal performance of our model, which can vary significantly over time. We will explore alternative evaluation metrics that offer a more nuanced understanding of our model's performance dynamics over extended periods.

Accuracy at a given event index is a performance metric used to assess the effectiveness of a predictive model at a specific position within the sequence of events. In mathematical terms, if we denote γ as the size of the average accuracy window, and $\hat{y}_j$ as the predicted activity label of an event at index j based on preceding events, with y_j representing the actual activity at index j , the accuracy at index i can be expressed as:

$$accuracy_i = \frac{1}{\gamma} \sum_{j=i-\gamma}^i 1\{\hat{y}_j = y_j\}. \quad (2)$$

The formula computes the ratio of correct predictions from j up to event index i , averaging the predictions in the window.

Average accuracy is calculated by averaging across all events to determine if each activity is predicted correctly. To calculate the average accuracy, we have the following formula:

$$accuracy = \frac{1}{\mathcal{N}} \sum_{j=0}^{\mathcal{N}} 1\{\hat{y}_j = y_j\}. \quad (3)$$

where $\mathcal{N}$ is the total number of events in the stream. In both cases, $1\{\cdot\}$ denotes the indicator function, yielding 1 if its argument holds and 0 otherwise.

Task-specific forgetting is the difference in the accuracy of predictions when the task is encountered for the first time compared to the accuracy during subsequent encounters. For each task and each appearance of a task, we compute the following:

$$task_accuracy(n, a) = \mathcal{R}_{t_n, t_{n1}} - \mathcal{R}_{t_n, t_{na}}. \quad (4)$$

where n is the task, $\mathcal{R}_{t_n, t_{ni}}$ is the accuracy of task t_n after encountering it for the i^{th} time, and $a > 1$ is the appearance of the subsequent task, except for the first task.

The **running time** is computed by taking the difference between the end time and the start time.

5.2 The Datasets

We provide an overview of the datasets utilized in our study, comprising a diverse selection of both real-world and synthetic datasets.

To generate sudden concept drift datasets, the following scheme was employed: for each concept, $\mathcal{N}$ examples are sampled. The samples of each concept are concatenated to form a new event log, alternating between the concepts. Consequently, a sudden concept drift is introduced at every $\mathcal{N}$ events.

To replicate the occurrence of a drift in a log, the authors of [13] created a reference set of 72 event logs by modifying various parameters. They systematically modified a base model by applying one of twelve simple change patterns. Using the Business Process Drift data, three datasets were generated, each with different lengths and characteristics. The first dataset, *RandomTasks*, consists of seven distinct tasks arranged in batches without any specific order. Each batch contains all tasks, and the entire dataset is made up of four batches, totaling 80,406 events. The second dataset, *ImbalancedTasks*, includes four tasks that occur with varying frequencies, with each task appearing between one and four times, totaling 30,754 events. The third dataset, *RecurrentTasks*, features four distinct tasks that recur within the same loop multiple times, totaling 100,739 events.

Moreover, we utilize a real dataset called *Recurrent BPIC2015*, which is a recurrent dataset generated from actual data collected across five different municipalities. Each task within *BPIC2015* corresponds to data from a specific municipality. This recurrent *BPIC2015* dataset comprises 32,016 events.

5.3 The Competitors

The model's performance is compared against five competitors, including two baseline methods: *Landmark* and *Incremental Update* ($w = \text{Last Drift}$). In the **Landmark** approach [14], the model retrains from scratch with each new data window. In the **Incremental Updating** ($w = \text{Last Drift}$) approach [14], the model updates based on historical data up to the last observed concept drift.

In addition to these, three state-of-the-art methods are used as competitors. The method **DynaTrainCDD** [9] distinguishes itself through its advanced concept drift detection algorithm called *PrefixTreeCDD*. It continually monitors process data for deviations and utilizes *Prefix Trees* to represent and analyze process sequences efficiently. These detected drifts dynamically dictate the frequency of updates and the selection of datasets for retraining. **TFCLPM** [20] dynamically updates its model based on a retraining dataset that combines recent events with hard samples. It is processed through a *Single Dense Layer* model using a dynamic loss function that incorporates *Mean Squared Error* and *Memory Aware Synapses* to mitigate significant parameter

changes. The approach using GANs [19] is constructed upon the principle of establishing a competitive game between two entities, each represented by a Recurrent Neural Network (RNN). Throughout the training process, one player gradually learns to generate event sequences that closely resemble those observed in the training data, while the other player evaluates the realism of this prediction.

To ensure a fair comparison, all approaches are adapted into a multi-layer self-attention model. Additionally, the model in the GAN-based approach is initially trained on 500 events and then continuously updated with each new window of events.

5.4 Parameter Selection

Three parameters are fine-tuned for CNAPwP. The `window_size` specifies the dimensions of the window, the `buffer_size` denotes the capacity of the buffer, and the `threshold` sets the criteria for deciding whether new data pertains to an existing task or necessitates the initialization of a new task. All parameter configurations are assessed by measuring the average accuracy of the resulting predictions and the running time. Table 1 lists the parameter values that yielded the best performance.

Table 1. The final parameter settings for all datasets.

Dataset	<code>window_size</code> γ	<code>buffer_size</code> ρ	<code>threshold</code> $\mathcal{E}$
RandomTasks	250	100	0.5
ImbalancedTasks	500	100	0.6
RecurrentTasks	250	150	0.6
Recurrent BPIC2015	250	100	0.8

5.5 Ablation Study

To confirm the effectiveness of using both the E- and G-prompts, an ablation study compares the average accuracy on the `RecurrentTasks` dataset under four conditions: using only the E-prompt, using only the G-prompt, using neither prompt, and using both prompts (CNAPwP). For the baseline with no prompts, a model is trained on the initial 500 events, after which its layer weights are frozen, leaving only the classification head trainable.

Table 2. Average accuracy value for each condition on the `RecurrentTasks` dataset.

Condition	CNAPwP	E-Prompt	G-Prompt	No Prompt
Average Accuracy	78.90%	77.30%	77.07%	70.86%

Table 2 displays the accuracy values for the four distinct conditions. The condition employing both the E- and G-prompts achieves the highest accuracy on the `RecurrentTasks` dataset. From this, we can conclude that using both prompts yields the best results.

Prompting Function. The original DualPrompt study compared two prompting functions: Prefix Tuning and Prompt Tuning [21]. We compared average accuracy and running time for each dataset and prompting function. Prefix Tuning generally had higher accuracy and lower running time. Consequently, Prefix Tuning was chosen for all further experiments.

5.6 Results

Table 3 provides a comprehensive overview of the average accuracies across all datasets and methods.

Table 3. Average accuracy and standard deviation (after 5 runs of each method over each dataset). **Bold** indicates the highest accuracy, *italic* indicates the second-highest accuracy, and underlined indicates the third-highest accuracy.

Method	Dataset			
	RandomTasks	ImbalancedTasks	Recurrent BPIC2015	RecurrentTasks
CNAPwP (Ours)	81.33% $\pm$ 0.39	79.66% $\pm$ 0.71	66.18% $\pm$ 0.31	78.90% $\pm$ 0.24
DynaTrainCDD [9]	74.65% $\pm$ 0.45	75.79% $\pm$ 0.63	54.27% $\pm$ 0.68	73.06% $\pm$ 0.40
TFCLPM [20]	<u>79.50%</u> $\pm$ 0.12	81.71 $\pm$ 0.44	<u>65.50</u> $\pm$ 0.16	<u>78.10</u> $\pm$ 0.08
GAN method [19]	73.94% $\pm$ 0.12	70.37% $\pm$ 0.43	60.41% $\pm$ 0.49	69.44% $\pm$ 0.09
Landmark [14]	79.97% $\pm$ 0.71	79.55% $\pm$ 0.66	67.00% $\pm$ 0.55	<u>77.99%</u> $\pm$ 0.20
Last drift [14]	75.97% $\pm$ 0.14	74.54% $\pm$ 0.75	57.01% $\pm$ 0.41	72.70% $\pm$ 0.20

The ImbalancedTasks dataset aims to investigate whether the frequency of task occurrences affects the method’s performance. Table 3 shows that TFCLPM outperforms all other methods with a notably higher average accuracy, demonstrating its effectiveness in scenarios with few tasks and no specific order. CNAPwP follows in second place, indicating an improvement over DynaTrainCDD and the GAN method. However, in Fig. 3 it shows that the frequency of task occurrences might influence the results for all approaches. In the figures illustrating accuracy by event index, each color represents a distinct task, highlighting both the number of tasks and their frequency of occurrence. Initially, all methods exhibit decreased accuracy upon task changes. Subsequently, none of the methods recover the high accuracy observed during the initial tasks, implying a loss of information from Tasks 1, 2, and 3. Task 4 notably impacts the methods, resulting in decreased performance for subsequent occurrences. TFCLPM initially achieves a higher accuracy than the other methods for the first three tasks but ultimately also declines to a lower accuracy. Thus, even though the average accuracy is the highest, the accuracies per event index are not significantly higher after the fourth task is introduced. CNAPwP demonstrates effective adaptation to changing tasks and maintains robust performance across tasks, with fewer accuracy drops compared to other methods.

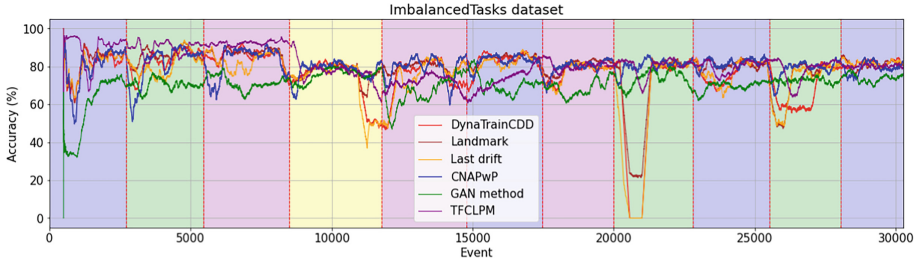


Fig. 3. The accuracy per event for all methods on the `ImbalancedTasks` dataset.

In Table 3, we observe that CNAPwP maintains the highest accuracy among all methods for the `RecurrentTasks` and the `RandomTasks` dataset. In Fig. 4 and Fig. 5, it can be seen that initially, the Last Drift method consistently shows the lowest accuracy declines upon task changes. DynaTrainCDD experiences milder drops in accuracy, but its recovery time is longer compared to CNAPwP. Although CNAPwP occasionally encounters minor accuracy decreases after task changes, it consistently recovers swiftly. The Landmark method manages to prevent accuracy drops following task changes. The method using GANs exhibits better performance on the `RecurrentTasks` dataset compared to other datasets, but it still faces challenges

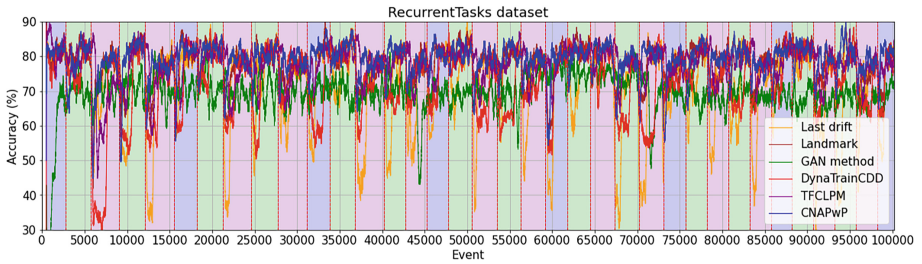


Fig. 4. The accuracy per event for all methods on the `RecurrentTasks` dataset.

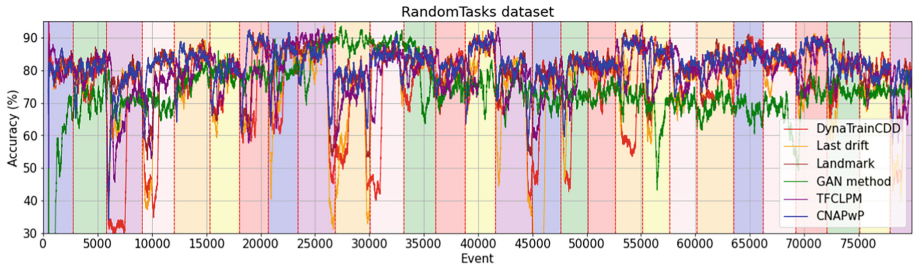


Fig. 5. The accuracy per event for all methods on the `RandomTasks` dataset.

after initialization. However, when it comes to the `RandomTasks` dataset, we observe a decline in accuracy as the occurrences of tasks increases.

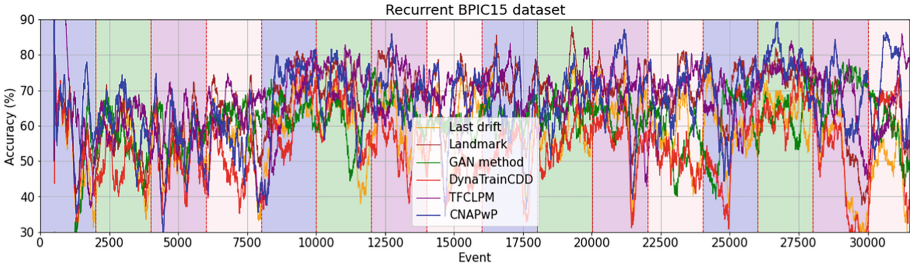


Fig. 6. The accuracy per event for all methods on the Recurrent BPIC2015 dataset.

Table 3 highlights the challenges encountered by all methods when applied to the Recurrent BPIC2015 dataset, which is known for its complexity with 181 unique events. This complexity poses a significant challenge to accurately predict the next activities. All methods take considerable time to achieve approximately 80% accuracy, a benchmark reached by other datasets. The Landmark method achieves the highest accuracy on this dataset, with CNAPwP closely following. In Fig. 6, it is evident that none of the methods maintain stable accuracy throughout the dataset; instead, they exhibit fluctuations following task changes and also during task execution. All methods perform comparably until the reintroduction of previously learned tasks, whereupon there is a noticeable increase in accuracy across all methods with the recurrence of Task 1.

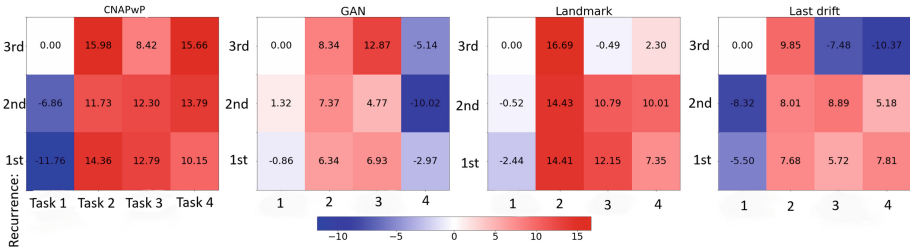


Fig. 7. The task-specific difference in the performance of four methods on the Recurrent BPIC2015 dataset for three recurrences w.r.t. the accuracy of the task first appearance. The x-axis contains the task id, and the y-axis contains the recurrence of each task.

In Fig. 7, task-specific difference in performance is illustrated. In this figure, it can be seen that after the initial occurrences of Tasks 2, 3, and 4, we observe significant increases in delta values for CNAPwP and Landmark. While the accuracy generally decreases for Task 1 across all methods during the data stream, CNAPwP is the only method where accuracy consistently improves for each subsequent task occurrence.

Table 4. Processing time per event and standard deviation after 5 runs of each method over each dataset (in milliseconds).

Method	Dataset			
	RandomTasks	ImbalancedTasks	Recurrent BPIC15	RecurrentTasks
CNAPwP (Ours)	3.29 ± 2.13	2.98 ± 1.42	25.04 ± 1.76	3.16 ± 2.34
DynaTrainCDD [9]	2.06 ± 1.24	1.90 ± 1.37	9.13 ± 1.84	2.45 ± 0.87
GAN [19]	5.08 ± 2.15	5.22 ± 2.79	22.50 ± 1.38	5.23 ± 1.95
Landmark [14]	63.85 ± 6.47	18.45 ± 8.32	229.25 ± 7.68	33.98 ± 9.13
Last drift [14]	3.03 ± 1.96	2.41 ± 1.52	21.26 ± 1.43	2.62 ± 0.93
TFCLPM [20]	2.23 ± 2.31	4.62 ± 2.65	12.14 ± 3.12	2.81 ± 1.72

To assess the feasibility of our approach for real-time implementation, we can calculate the processing time for each event, shown in Table 4. DynaTrainCDD consistently demonstrates the lowest running time per event across all datasets. Generally, CNAPwP does not emerge as the fastest method. The more complex the dataset, the longer CNAPwP takes to execute. Nevertheless, the processing time per event remains relatively low, keeping the approach close to real-time performance.

6 Conclusion and Outlook

In this work, we propose CNAPwP, a novel approach for continual next-activity prediction aimed at mitigating catastrophic forgetting. We evaluated its performance using various metrics, including a novel measure called task-specific forgetting, which quantifies the method’s memory retention. Experiments conducted on several datasets indicated that CNAPwP achieves the highest average accuracy on two of the four datasets. In one dataset, it ranks second, with Landmark performing the best but being highly inefficient. Therefore, CNAPwP is still regarded as the superior option. Furthermore, analyses of accuracy per event index and task-specific forgetting indicate that our approach exhibits minimal forgetting compared to alternative methods.

Detecting the concept drifts was outside the scope of the paper. In the future, it is crucial to explore areas such as integrating a robust concept drift detection mechanism, possibly for both sudden and gradual drifts. Additionally, exploring alternative methods for task comparison might offer sharper, faster and more efficient way of the task similarity detection part.

References

1. Chrysakis, A., Moens, M.F.: Online continual learning from imbalanced data. In: International Conference on Machine Learning, pp. 1952–1961. PMLR (2020)
2. Feldman, Z., Fournier, F., Franklin, R., Metzger, A.: Proactive event processing in action: a case study on the proactive management of transport processes. In: DEBS, pp. 97–106 (2013)

3. Ferilli, S., Angelastro, S.: Activity prediction in process mining using the WoMan framework. *J. Intell. Inf. Syst.* **53**(1), 93–112 (2019). <https://doi.org/10.1007/s10844-019-00543-2>
4. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. *ACM Comput. Surv. (CSUR)* **46**(4), 1–37 (2014)
5. Hou, S., Pan, X., Loy, C.C., Wang, Z., Lin, D.: Lifelong learning via progressive distillation and retrospection. In: *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 437–452 (2018)
6. Huete, J., Qahtan, A.A., Hassani, M.: PrefixCDD: effective online concept drift detection over event streams using prefix trees. In: *COMPSAC*, pp. 328–333 (2023)
7. Hurtado, J., Salvati, D., Semola, R., Bosio, M., Lomonaco, V.: Continual learning for predictive maintenance: overview and challenges. *Intell. Syst. Appl.* **19**, 200251 (2023)
8. Kirkpatrick, J., et al.: Overcoming catastrophic forgetting in neural networks. *CoRR abs/1612.00796* (2016)
9. Kosciuszek, T., Hassani, M.: Online next activity prediction under concept drifts. In: *Advanced Information Systems Engineering Workshops*, pp. 335–346 (2024)
10. Lesort, T., Caselles-Dupré, H., Garcia-Ortiz, M., Stoian, A., Filliat, D.: Generative models from the perspective of continual learning. In: *IJCNN*, pp. 1–8. *IEEE* (2019)
11. Li, Z., Hoiem, D.: Learning without forgetting. *IEEE Trans. Pattern Anal. Mach. Intell.* **40**(12), 2935–2947 (2017)
12. Lopez-Paz, D., Ranzato, M.: Gradient episodic memory for continual learning. *Adv. Neural. Inf. Process. Syst.* **30**, 6470–6479 (2017)
13. Maaradji, A., Dumas, M., La Rosa, M., Ostovar, A.: Fast and accurate business process drift detection. In: *BPM*, pp. 406–422 (2015)
14. Pauwels, S., Calders, T.: Incremental predictive process monitoring: the next activity case. In: *BPM*, pp. 123–140 (2021)
15. Rahaf, A., Lucas, C.: Online continual learning with maximally interfered retrieval. In: *NIPS* (2019)
16. Rebuffi, S.A., Kolesnikov, A., Sperl, G., Lampert, C.H.: iCaRL: incremental classifier and representation learning, pp. 5533–5542 (2017)
17. Rolnick, D., Ahuja, A., Schwarz, J., Lillicrap, T., Wayne, G.: Experience replay for continual learning. In: *Advances in Neural Information Processing Systems*, vol. 32 (2019)
18. Rusu, A.A., et al.: Progressive neural networks (2016)
19. Taymouri, F., Rosa, M.L., Erfani, S., Bozorgi, Z.D., Verenich, I.: Predictive business process monitoring via generative adversarial nets: the case of next event prediction. In: *BPM*, pp. 237–256 (2020)
20. Verbeek, T., Yao, R., Hassani, M.: Task-free continual learning with dynamic loss for online next activity prediction. In: *ICPM Workshops* (2024, to appear)
21. Wang, Z., et al.: DualPrompt: complementary prompting for rehearsal-free continual learning. In: *ECCV*, pp. 631–648 (2022)
22. Wang, Z., et al.: Learning to prompt for continual learning. *CoRR* (2022)
23. Wolters, L., Hassani, M.: Predicting activities of interest in the remainder of customer journeys under online settings. In: *ICPM Workshops*, pp. 145–157 (2022)
24. Zenke, F., Poole, B., Ganguli, S.: Continual learning through synaptic intelligence. In: *International Conference on Machine Learning*, pp. 3987–3995. *PMLR* (2017)